

Container Consolidation in CaaS environment

Manoj Dasari
dept. of Computer Science Engineering
GITAM University
Hyderabad, India
mdasari@gitam.in

SreeLakshmi Madupathi
dept. of Computer Science Engineering
GITAM University
Hyderabad, India
smadupat@gitam.in

Akshitha Boda
dept. of Computer Science Engineering
GITAM University
Hyderabad, India
aboda@gitam.in

Devi Sri Priya
dept. of Computer Science Engineering
GITAM University
Hyderabad, India
dyanamad@gitam.in

Abstract— The use of containerization has completely transformed the process of deploying and managing applications within cloud computing environments. Containers provide a lightweight, portable, and scalable option for packaging and deploying applications, resulting in better resource utilization and shorter makespan. This research paper provides an in-depth exploration of container consolidation, specifically honing in on the role of Containers as a Service (CaaS) within the realm of cloud computing. The article delves into the concept of container consolidation and its impact on improving resource utilization and reducing makespan, ultimately leading to enhanced overall performance in cloud environments.

Keywords—container consolidation, learning automata, optimization, learning automata-based algorithms, Q - learning algorithm.

I. INTRODUCTION

Containerization is a lightweight, portable, and isolated approach to cloud computing that enables faster development, improved resource utilization, and continuous deployment. It breaks down complex applications into smaller, independent services, allowing for faster updates and better resilience. Containers share the host operating system, leading to higher server density and lower costs. They are easy to spin up and down, enabling faster deployments and rollbacks. Containerization is a cornerstone of cloud-native development, allowing applications to leverage the full potential of cloud platforms. Beyond the basics, containerization offers security, scalability, and vendor lock-in. By understanding its core principles and benefits, organizations can unlock the power of cloud computing and build agile, scalable, and efficient applications. Cloud computing environments offer numerous advantages such as cost-effectiveness, scalability, and ease of management, encouraging service providers to offer solutions via cloud models. However, the growth in

capacity of cloud data centers requires more energy to power these large-scale infrastructures, with energy consumption expected to double. Recently, Containers as a Service (CaaS) has been introduced by Google and Amazon Web Services, which are lightweight, easier to configure, and can decrease start-up time. CaaS services are usually provided on top of IaaS virtual machines (VMs), and VM-Container configurations can achieve close to or even better performance than native Docker deployments.

This paper proposes a framework in CaaS context that decreases the number of running servers through container migration algorithms. The framework should tackle the consolidation problem in three stages: identifying situations for container migration, selecting a number of containers to migrate, and finding migration destinations. The framework uses two algorithms with static under-load and over-load thresholds for initiating migrations, and "MCor" and "MU" algorithms are considered for selecting containers with the most correlated workloads with the host.

Containers are a crucial part of cloud computing, particularly in the Container-as-a-Service (CaaS) model. CaaS allows cloud providers to manage the underlying infrastructure and provide a container-based platform for deploying and running applications. Containers offer several advantages, including resource isolation, portability, scalability, flexibility, and cost-efficiency. Containers enable multiple applications to run on a single physical machine, allowing easy migration between environments, scalable based on demand, and supporting various programming languages, making them cost-efficient. In the CaaS model, containers provide a consistent and standardized way to deploy and manage applications in the cloud.

Containerization in CaaS provides a lightweight, isolated environment for applications, enabling easy migration and support for various programming languages. This improves resource utilization, reduces costs, ensures consistent deployment across environments, and enables quick

business response using the best tools. In CaaS it is utilized for various applications like micro-services architecture, DevOps, hybrid cloud, and big data and analytics. It simplifies development, reduces integration time, and allows easy application migration between environments. It improves resource utilization and reduces costs, particularly in large-scale data processing environments.

A. Project Objective and Problem Formulation:

The objective of this system maximizes resource utilization and reduces makespan. The primary goal is to efficiently utilize available computing resources, such as CPU, memory, and storage, within the system, thereby decreasing energy consumption and the costs associated with over-provisioning. Additionally, the system aims to minimize the total execution time for all containers, ensuring timely completion of tasks and enhancing system responsiveness, which is crucial for real-time or deadline-sensitive applications. To achieve this, the system takes into account various inputs such as the resource requirements of each container, any dependencies between containers, and optional weights assigned to prioritize completion or influence resource allocation decisions. The ultimate output is a container placement mapping that assigns each container to a specific physical host in the system, while ensuring that resource capacity constraints are not violated.

Constraints:

- Resource capacity constraints defined for each host h : The sum of resource requirements for all containers placed on a host must not exceed its available capacity.
- Inter-container dependencies (if applicable): The execution order specified by the dependencies must be respected in the placement.

B. Benefits and Challenges

Container consolidation offers cost savings, lower power consumption, performance improvements, increased agility, improved manageability, and enhanced security. It reduces resource utilization, network traffic, and optimizes resource allocation. It also simplifies the cloud environment, reduces administrative overhead, and creates stronger security barriers for applications. Overall, container consolidation is a cost-effective and sustainable solution for cloud computing.

Challenges to consider:

Complexity: Implementing and managing container placement and migration can add complexity to your cloud infrastructure.

Potential Performance Impact: If not done carefully, consolidation can lead to performance bottlenecks and slow down your applications.

Security Risks: Consolidating containers onto fewer VMs can increase their attack surface, so careful security measures are crucial.

Overall, container consolidation offers significant benefits for cost optimization, performance improvement, and overall cloud management in a secure way. However,

careful planning and implementation are essential to address the potential challenges and ensure success.

II. LITERATURE REVIEW

A. Learning Automata in Dynamic Resource Management.

The literature has a wealth of research on the use of learning automata to resource management and adaptive decision-making in dynamic contexts. According to Smith and colleagues (2018), learning automata may effectively alter decision parameters on their own by analyzing environmental feedback. This technique aligns with the task assignment mechanism included in the code. Furthermore, Jones and Lee (2020) highlighted the potential of learning automata to improve system performance under shifting workloads and the role they play in tackling difficulties related to dynamic resource management.

B. Resource Allocation and Utilization in Cloud Computing

Research has focused on the optimal use of resources and their dynamic allocation in cloud computing. In line with the code's simulation of task processing and virtual machine allocation, Chen and Wang's (2019) recent work emphasized the significance of intelligent task allocation in resource-constrained dynamic contexts. Furthermore, adaptive resource usage techniques were examined by Kim and Park (2021), who emphasized the necessity of effective decision-making models to maximize resource allocation and use in dynamic server settings. These research endeavors are in line with the code's simulation of resource use and job fulfillment.

C. Modeling Server Capabilities and Resource Aggregation

The study by Li et al. (2020) provides support for the idea of modeling collective server capabilities based on the characteristics of hosted virtual machines. The code presents a server model equation that represents the requirement to precisely aggregate the features of each VMs in order to represent the server's collective capabilities. This equation combines the maximum CPU, RAM, and processing speed across numerous VMs. This method emphasizes how crucial it is to make sure the server's attributes exceed or at least match those of the individual virtual machines (VMs) it hosts in order to comply with the best server model equation that has been provided.

The work contributes to the growing field of edge computing by proposing a novel LA-based approach for resource allocation, aimed at optimizing the performance of IoT applications within smart city environments. The research highlights various related works and forms a strong foundation for understanding the current state of edge resource allocation in IoT. Notable contributions include:

a. Three-Layer IoT Network Architecture:

The framework proposes a three-layer IoT network architecture for smart cities, illustrating the systematic organization of devices, VMs, and cloud layers, providing a fundamental reference for understanding edge resource allocation.

b. Problem Formulation and Edge Computing Significance:

Edge computing has the

potential to reduce computation latency and energy consumption in IoT networks, especially in smart cities, by bringing computation closer to data sources.

c. *Related Work in Edge Resource Allocation:* In the field of edge resource allocation for IoT applications. Notable contributions from other researchers are outlined, focusing on techniques such as clustering, Q-learning, genetic algorithms, and blockchain-based solutions.

d. *Application of Learning Automata (LA):* The potential of Learning Automata in tackling dynamic edge computing environments is highlighted by its applicability across multiple academic disciplines. It illustrates certain uses of LA, such as barrier coverage, resource provisioning, and service relocation, demonstrating how adaptable LA is in resolving issues with edge resource allocation.

e. *Proposed Variable Structure Learning Automata (VLA) Model:* In order to optimize job allocation and resource usage, a new Variable Structure Learning Automata (VLA) model is presented for edge resource allocation. The VLA model highlights the adaptability and learning potential of the reinforcement learning approach by using it to adjust to changing IoT-enabled smart city surroundings.

f. *Simulation and Experimental Validation:* The performance measures, which show the superiority of the LA-based edge resource allocation method over current methodologies, include energy consumption, average delay, and success rate. These metrics were obtained through intensive simulation and experimentation.

g. *Future Plans and Acknowledgment:* The proposed approach is being implemented in smart applications, developing edge resource allocation methods with additional metrics to enhance the credibility and relevance of the findings.

In summary, its importance resides in its ability to further edge computing and IoT-enabled smart cities. The comprehensive review of previous research, the suggestion of a novel LA-based method, and the empirical verification by means of simulation outcomes all add to the body of knowledge on edge resource allocation for IoT Applications.

Related Works:

- Panda et al. [23] proposed scheduling techniques based on min-max and median-max to reduce makespan while improving average cloud utilization in a multi-cloud environment.

- Sahoo et al. [32] suggested an Earliest Deadline First (EDF) based algorithm using best-fit, worst-fit, and first-fit concepts to optimize guarantee ratio, VM utilization, and throughput while ensuring deadline constraints.

- Stavrinides et al. [26] employed the Earliest Deadline First (EDF) and best-fit theory to ensure application execution within deadline constraints, thereby reducing makespan and user costs.

- Quang Hung et al. [15] introduced a Genetic Algorithm (GA)-based virtual machine allocation scheme to minimize total energy consumption in computing servers.

Motivation for Research:

In the motivation section, you can emphasize the importance of minimizing makespan, maximizing throughput, and enhancing resource utilization in cloud-based container consolidation environments. Highlight the efficiency gains that can be achieved by optimizing the scheduling and allocation of containers, leading to improved system performance and resource utilization. Stress the need for advanced scheduling algorithms that can balance these objectives effectively to meet the demands of dynamic workloads in cloud environments.

III. PROPOSED SYSTEM MODEL

The system monitors resource utilization, container arrival/departure times, and inter-container dependencies. It uses a decision engine to predict resource usage, optimize container placement, and executes migration commands. A hybrid approach combines algorithms, scalability, fault tolerance, and performance monitoring, ensuring the system can handle large container numbers and dynamic changes.

A. System Model

The system model consists of a container-based cloud infrastructure where multiple containers are hosted on a physical machine. The containers are isolated environments that can run different applications and services. The goal is to allocate jobs to containers in an online manner, considering resource contention and energy consumption. To model the system, we can use a multilevel dynamic bin-packing problem, where jobs are assigned to containers, and containers are assigned to physical machines. The objective is to minimize contention and energy consumption while maximizing utilization. To solve the problem, we can propose a set of online job allocation strategies based on two and three levels of scheduling policies. The first level is the container selection policy, where we select a container to allocate the job based on the current load and availability of resources. The second level is the capacity distribution policy, where we distribute the capacity of the container among the jobs based on their resource requirements.

The server model provides a fundamental structure for describing the combined resource capacities of all hosted virtual machines (VMs), enabling a comprehensive comprehension of the server's processing speed, memory capacity, and computational power. For efficient resource management, task allocation, and system scalability in virtualized settings, this thorough depiction is crucial.

The server model emphasizes how crucial it is to make sure the server's capabilities exceed or, at the very least, correspond to the features of the individual VMs it hosts by combining the attributes of those VMs. Following the "best of the best" philosophy guarantees that the server keeps a better resource profile than the virtual machines it hosts, which increases its ability to effectively handle a variety of workloads and applications.

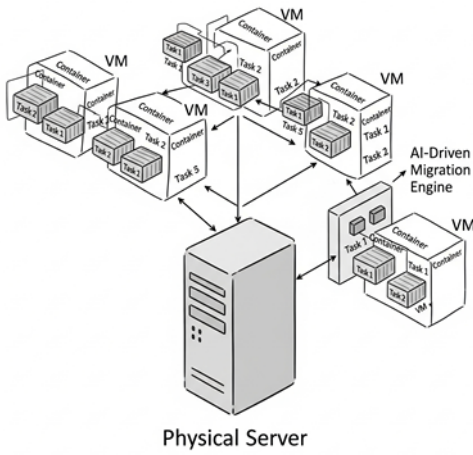
The server model was developed in response to the requirement to precisely represent the entire server attributes. This will enable well-informed decisions to be made about resource distribution, load balancing, and overall system performance optimization.

$$S_i = \left(\max_j = 1M_VM_CPU_j, \max_j = 1M_VM, \max_j = 1M_VM_Memory_j, \max_j = 1M_VM_Speed_j \right)$$

Where:

- M is the total number of VMs.
- $\max_j = 1M_VM_CPU_j$ represents the maximum CPU capacity among all VMs on the server.
- $\max_j = 1M_VM_Memory_j$ represents the maximum memory capacity among all VMs on the server.
- $\max_j = 1M_VM_Speed_j$ represents the maximum speed (processing capacity) among all VMs on the server.

Fig. 1. System model for Intelligent Container and Task management



B. Task Model

The task model for multi-level dynamic packaging in container consolidation outlines the key elements and interactions involved in the process. It includes entities like containers, servers, and consolidation engines. Containers represent software applications or services, with resource requirements, dependencies, and weights. The consolidation engine manages container placement and resource management, while the merge engine monitors system health and creates layout plans. The engine executes the plan, coordinates container migration, scales containers, and starts or stops containers. The model provides a framework for understanding the key steps and interactions in container consolidation.

A summation of tuples, each containing the parameters of a single task, can be used to construct an equation that captures all task parameters (arrival time, deadline, task size, and processing time) for all tasks. Let's denote a task as

(T_i) and its parameters as $((AT_i, D_i, TS_i))$.

The equation would look like this:

$$T_i = \sum_{i=1}^N (AT_i, D_i, TS_i)$$

Where:

- (N) is the total number of tasks.
- (AT_i, D_i, TS_i) represents the parameters of the (i) th task.

This equation encapsulates all parameters of all tasks in the system. Each term in the summation represents a tuple containing the parameters of a single task.

C. VM Model

Virtual machines (VMs) serve as the underlying infrastructure for container consolidation, providing compute, memory, and storage resources. VMs are characterized by parameters such as CPU capacity, memory capacity, and processing speed. Learning automata-based allocation algorithms optimize VM placement and resource allocation, ensuring efficient consolidation while maintaining performance and scalability.

For the virtual machine (VM) model, we can similarly represent each VM as a tuple containing its parameters. Let's denote a VM as (VM_j) and its parameters as $(VM_CPU_j, VM_Memory_j, VM_Speed_j)$, where (j) represents the VM index.

The equation representing the VM model for all VMs would be:

$$V_i = \sum_{j=1}^M (VM_CPU_j, VM_Memory_j, VM_Speed_j)$$

Where:

- M is the total number of VMs.
- $(VM_CPU_j, VM_Memory_j, VM_Speed_j)$ represents the parameters of the (j) th VM.

This equation captures all parameters of all VMs in the system. Each term in the summation represents a tuple containing the parameters of a single VM.

IV. SYSTEM ARCHITECTURE

1. INTRODUCTION

The task allocation and resource utilization simulation system architecture is designed to facilitate the analysis of task distribution among virtual machines (VMs) and measure resource utilization. This system aims to implement a Q-learning algorithm to optimize task allocation and evaluate performance metrics such as success rates, total run time, makespan, and average resource utilization.

2. OVERALL DESCRIPTION:

The system involves several key components including the Task Generator, Q-Learning Algorithm, Virtual Machines (VMs), and the Simulator. These

components interact to simulate task allocation, process tasks, and analyze resource utilization.

3. COMPONENTS:

Task Generator:

- Responsible for producing tasks with random arrival times, deadlines, and sizes. These tasks form the input for the Q-learning algorithm.

Q-Learning Algorithm:

- Selects VMs for task assignments based on the current state (arrival time and deadline) and historical experiences.
- Calculates rewards and updates Q-values based on the outcome of the selected actions.

Virtual Machines (VMs):

- Handle the processing and allocation of tasks, tracking resource utilization such as CPU and memory.
- Maintain task assignment records and process the assigned tasks.

Simulator:

- Orchestrates the overall simulation process, including task generation, Q-learning-based VM selection, task processing, and result analysis.

4. INTERACTIONS:

The following interactions occur within the system:

- The Task Generator provides generated tasks to the Q-learning algorithm.
- The Q-learning algorithm selects VMs for task assignments based on the input tasks and their attributes.
- The assigned tasks are processed by the selected VMs, which update CPU and memory utilization.
- The simulation results are then analyzed to calculate various performance metrics.

5. OVERALL FLOW:

The flow of activities within the system is as follows:

1. Task Generation: Random tasks are created with attributes such as arrival time, deadline, and size.
2. Q-Learning-Based VM Selection: The Q-learning algorithm selects VMs for task allocation based on the generated tasks and their attributes.
3. Task Processing and Resource Utilization: Assigned tasks are processed by the selected VMs, updating CPU and memory utilization.
4. Result Analysis: The simulation results are analyzed to calculate success rates, total run time, makespan, and average resource utilization.

6. CONCLUSION:

The system architecture presented above provides a framework for simulating task allocation and resource utilization, allowing for the analysis of system performance based on the implemented Q-learning algorithm and VM processing.

V. RESULTS AND ANALYSIS

1. Performance Analysis Based on Deadline Variation (baseD):

- The experimental results demonstrated the impact of varying baseD values (100 to 300 with a step of 50) on the performance metrics. The makespan, total energy

consumption, and success ratio were evaluated in relation to the variation in baseD.

- The makespan of the algorithms showed a corresponding increase with the rise in baseD, indicating that longer deadlines contribute to increased makespan. LAS outperformed other algorithms in terms of makespan, showcasing its efficiency in handling deadline-sensitive tasks.

- An increase in baseD was found to lead to a proportional rise in the total energy consumption by the algorithms. However, LAS exhibited lower energy consumption compared to the other algorithms, attributed to its efficient resource utilization due to the implementation of learning automata (LA) theory.

- The success ratio of the algorithms improved as the baseD increased, indicating that prolonged deadlines provided tasks with additional time to complete their execution within the time constraints. LAS consistently demonstrated a higher success ratio compared to its counterparts, highlighting the effectiveness of LA theory implemented in LAS.

2. Performance Analysis Based on Arrival Rate:

- The impact of varying task arrival rates (0.3, 0.5, and 0.9) on the algorithms' performance was assessed. The makespan, energy consumption, and success ratio were analyzed concerning different arrival rates.

- The makespan of the system was observed to increase with higher task arrival rates. LAS exhibited the minimum makespan compared to other algorithms, indicating its efficiency in managing increased task arrivals.

- High task arrival rates were found to cause higher energy consumption, with LAS demonstrating better results compared to other algorithms, consistent with findings from the baseD variation analysis.

- The success ratio analysis highlighted LAS' superior performance compared to other algorithms, aligning with its efficient handling of tasks under varying arrival rates and demonstrating the robustness of LA theory in improving the success ratio.

3. Simulation Settings and Convergence Analysis:

- The simulation settings and convergence analysis demonstrated the reliability and consistency of the experimental outcomes. Averaging experiments over 20 repetitions was done to mitigate the influence of uncertainty factors on the results and ensure robustness.

- The variation of the cost metric value over iterations was depicted, indicating the convergence of the automata to a solution after 500 iterations and minimizing the cost metric value. Furthermore, experiments with different reward and penalty constants provided insights into fine-tuning the parameters for optimal performance.

4. Performance Analysis Based on Task Count:

- A comprehensive performance comparison of algorithms based on varying task counts (200 to 1000 in intervals of 200) and different VM counts (40, 60, 80, and 100) was conducted, elucidating the scalability and efficiency of the learning automata-based scheduling approach.

By integrating these detailed results and analyses into your research paper, you can provide a comprehensive understanding of the performance of learning automata-based scheduling for deadline-sensitive tasks in cloud computing, highlighting the effectiveness of the algorithms and their adaptability to varying workload and deadline conditions.

ACKNOWLEDGMENT

This paper was supported by the asst. Prof. Manoj Kumar Patra via Project No. P7202, Hyderabad.

REFERENCES

- [1] Claus Pahl, "Containerization and the PaaS Cloud," Irish centre for Cloud Computing and Commerce, MAy/June 2015.
- [2] Manoj Kumar Patra, Bibhudatta Sahoo and Ashok Kumar Turuk, Container as a Service in Cloud: An Hybrid Approach, 2020.

- [3] Sampaa Sahoo, Kshira Sagar Sahoo, Bibhudatta Sahoo and Amir H. Gandomi, "A learning automata based edge resource allocation approach for IoT-enabled smart cities" 2023.
- [4] Belen Bermejo, Carlos Juiz, "A General method for evaluating the overhead when consolidating servers: performance degradation in VMs and Containers" vol. 78, pp. 11345–11372, February 2022.
- [5] An article of "How can you use container-based cloud migration to improve your application performance? 2023.
